

LocalBuka Intern Case Study

Track: Backend Engineering

Time Limit:	3-5 Days (Take-Home)
Starter Repo:	https://github.com/TobiAkinkuotu/assessment-repo
Submission:	Email your public fork link & 2-min Loom to hellohr@localbuka.com

Context Overview

In this backend track, you will build a clean, type-safe API matching LocalBuka's core restaurant discovery and points ecosystem. You will implement standard controllers, entities, services, and database migrations.

Step-by-Step Engineering Flow

Phase 1: Database & Seeding Setup

Initialize your database schema using PostgreSQL. Utilize the provided mock_data.json to seed tables. Do not alter raw naming shapes. Design tables for:

- **restaurants:** id (string), name (string), cuisine (string), price_range (int), latitude (double), longitude (double), address (text), tags (text array), avg_rating (decimal), is_open (int, 0/1).
- **users:** id (string), name (string), email (string), referred_by (string, nullable reference).
- **reviews:** id (string), restaurant_id (foreign key), user_id (foreign key), rating (int, 1-5), comment (text), tags (text array).
- **points_ledger:** id (string), user_id (foreign key), action (string), restaurant_id (nullable foreign key), points (int).

Phase 2: Core API Endpoints Implementation

Configure controllers and request handlers for the following specific paths:

- **GET /restaurants:** Supports search/filters. Query parameters: latitude (float), longitude (float), cuisine (string), priceRange (number), minRating (float). Must return JSON payload sorted closest to user coordinates if location parameters are provided.
- **GET /restaurants/:id:** Returns single profile object plus embedded reviews child relation list.
- **POST /restaurants/:id/reviews:** Add review. Request body: { userId: string, rating: number, comment: string, tags: string[] }. Validate rating input boundaries. Enforce a database/service block prohibiting users from leaving duplicate reviews.

Phase 3: Points Module & Business Logic Rules

Create logic handlers for transaction checks:

- **POST /points/earn:** Awards points based on action. Request payload: { userId: string, action: 'check-in' | 'review' | 'referral', restaurantId?: string }. Restrict check-in earnings to 1 check-in per user per restaurant per calendar day.
- **GET /points/balance/:userId:** Return current aggregate sum of points along with ledger record logs of all transactions.

Technical Assessment Questions

Q1. API Payload Parser & Types validation

Write a custom TypeScript parser function `parseRestaurant(raw: unknown): Restaurant`. It should transform and cast raw incoming payload properties from snake_case strings/floats to typed values (camelCase, numbers). Detail how to hook this validation into NestJS using class-validator DTOs.

Q2. Race Condition Prevention

Suppose a user runs a concurrent loop request calling points redemptions simultaneously. Explain one database-level lock strategy (e.g. SELECT FOR UPDATE or optimistic version checks) to guarantee safety from double redemptions.

Grading & Evaluation Rubric

Submissions are evaluated out of 100 points based on functional completeness and architecture:

Criteria	Weight	Description
API Endpoint Functionality	35 pts	Correct routes mapping, search query filtering logic, and correct HTTP response codes.
Business Logic Constraints	25 pts	Strictly blocks duplicate reviews and duplicate daily check-in points updates.
Database & Geolocation Math	20 pts	Correct Postgres schema setup and Haversine formula calculation for coordinates.
TypeScript Quality & DTOs	20 pts	Rigorous type casting, DTO validation usage, and absence of raw 'any' types.